Modelo Relacional — Agrogestión

1. Introducción

El presente documento describe en detalle el modelo relacional de la base de datos del sistema **Agrogestión**, una aplicación web SPA (*Single Page Application*) de gestión agrícola desarrollada como **Trabajo de Fin de Grado (DAW)**. El sistema permite a empresas agrícolas gestionar de forma integral sus operaciones: desde la administración de terrenos y la asignación de tareas hasta la comunicación entre los distintos roles del equipo de trabajo.

Stack tecnológico del backend:

- **Base de datos:** PostgreSQL 15+ (alojada en Supabase)
- **Autenticación:** Supabase Auth (JWT + `auth.users`)
- **Seguridad:** Row Level Security (RLS) nativo de PostgreSQL
- **Automatización:** Triggers PL/pgSQL + funciones edge (`pg_net`) para envío de emails
- **API:** Generada automáticamente por PostgREST (Supabase)

La base de datos consta de **10 tablas** organizadas en cuatro categorías funcionales, con **15 claves foráneas** que establecen las relaciones entre ellas. El diseño sigue principios de normalización (3FN) y emplea patrones como *soft delete* para preservar la integridad histórica de los datos.

---

2. Categorías de tablas

El modelo se estructura por colores en el diagrama adjunto (`Modelo_Relacional.drawio`):

Color	Categoría	Tablas	Descripción
🔵 **Azul**	Catálogos	`rol`, `estados_tarea`	Tablas de referencia con valores predefinidos. Raramente modificadas.
🟢 **Verde**	Entidades principales	`perfiles`, `tarea`, `terreno`	Tablas de negocio que almacenan los datos centrales del sistema.
🟠 **Naranja**	Tablas de relación (N:M)	`gerente_capataz`, `tarea_trabajador`, `capataz_trabajador`	Tablas intermedias que resuelven relaciones muchos-a-muchos mediante claves primarias compuestas.
🟣 **Morado**	Funcionalidades complementarias	`comentarios_tarea`, `notificaciones`	Tablas de apoyo que permiten la comunicación y el seguimiento dentro del sistema.

Convenciones del diagrama

Símbolo	Significado
🔑	Clave primaria (PK)
🔗	Clave foránea (FK) — las filas FK aparecen destacadas en amarillo
Línea **continua** gruesa	Relación obligatoria (FK NOT NULL)
Línea **discontinua**	Relación opcional (FK NULLABLE)

Terminación –||–	Cardinalidad **uno**
Terminación –<<	Cardinalidad **muchos**

---

3. Descripción de cada tabla

3.1. ROL (catálogo)

Almacena los 4 roles de usuario del sistema. Es una tabla de referencia que define la jerarquía de permisos de la aplicación.

Columna	Tipo	Restricciones
`id_rol`	INTEGER	**PK**
`nombre`	VARCHAR(50)	UNIQUE
`descripcion`	VARCHAR(255)	—

Valores predefinidos:

id_rol	nombre	Nivel de acceso
1	ADMIN	Acceso total al sistema, gestión de todos los usuarios
2	GERENTE	Gestión de terrenos, tareas y asignación de capataces
3	CAPATAZ	Supervisión de trabajadores y gestión de tareas asignadas
4	TRABAJADOR	Visualización y ejecución de tareas asignadas

> ***Regla de negocio:*** *Un usuario solo puede tener un rol a la vez. El rol determina qué datos puede ver y modificar gracias a las políticas RLS.*

3.2. ESTADOS_TAREA (catálogo)

Define los posibles estados por los que pasa una tarea a lo largo de su ciclo de vida.

Columna	Tipo	Restricciones
`id_estado`	SERIAL	**PK**
`nombre`	VARCHAR(50)	UNIQUE
`descripcion`	VARCHAR(255)	—

Valores predefinidos y flujo de transición:

```
PENDIENTE → ASIGNADA → ACEPTADA → EN_PROGRESO → COMPLETADA
                    ↘ RECHAZADA (el capataz puede rechazar la tarea)
```

id_estado	nombre	Descripción

1	PENDIENTE	Tarea creada pero no asignada a capataz
2	ASIGNADA	Tarea asignada a un capataz, esperando aceptación
3	ACEPTADA	El capataz ha aceptado la tarea
4	RECHAZADA	El capataz ha rechazado la tarea
5	EN_PROGRESO	La tarea está en ejecución por los trabajadores
6	COMPLETADA	La tarea ha sido finalizada

> ***Regla de negocio:*** *Solo el gerente puede crear y asignar tareas. Solo el capataz puede aceptar o rechazar. El estado cambia automáticamente según las acciones de cada rol.*

3.3. PERFILES (entidad principal)

Tabla central del sistema. Extiende la tabla interna `auth.users` de Supabase con los datos personales y profesionales de cada usuario. Cada registro se crea automáticamente cuando un usuario se registra, mediante el trigger `handle_new_user()`.

Columna	Tipo	Restricciones	Descripción
`id`	UUID	**PK**, FK → `auth.users`	Identificador de Supabase Auth
`nombre`	VARCHAR(100)	NOT NULL	Nombre del usuario
`apellidos`	VARCHAR(100)	NOT NULL	Apellidos del usuario
`dni`	VARCHAR(9)	UNIQUE, NOT NULL	Documento Nacional de Identidad (formato español)
`email`	VARCHAR(100)	UNIQUE, NOT NULL	Correo electrónico, sincronizado con `auth.users`
`tlf`	VARCHAR(15)	NOT NULL	Teléfono de contacto
`direccion`	VARCHAR(255)	NOT NULL	Dirección postal
`id_rol`	INTEGER	FK → `rol(id_rol)`	Rol asignado al usuario
`fecha_baja`	DATE	—	Fecha de desactivación (soft delete)
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de alta en el sistema

> ***Trigger automático:*** *Cuando un usuario completa el registro en Supabase Auth, el trigger `handle_new_user()` crea automáticamente un registro en `perfiles` vinculado por UUID, copiando el email y aplicando el rol seleccionado durante el formulario de registro.*

> ***Índices implícitos:*** *`id` (PK), `dni` (UNIQUE), `email` (UNIQUE), `id_rol` (FK).*

3.4. TAREA (entidad principal)

Tabla central de la operativa. Representa las tareas agrícolas creadas por los gerentes y asignadas a capataces, quienes a su vez designan trabajadores para su ejecución.

Columna	Tipo	Restricciones	Descripción

`id_tarea`	SERIAL	**PK**	Identificador autoincremental
`nombre`	VARCHAR(200)	NOT NULL	Título descriptivo de la tarea
`descripcion`	TEXT	—	Instrucciones detalladas
`fecha_inicio`	DATE	NOT NULL	Fecha planificada de inicio
`fecha_fin`	DATE	NOT NULL	Fecha planificada de finalización
`id_terreno`	INTEGER	FK → `terreno(id_terreno)`	Terreno donde se ejecuta
`id_gerente`	UUID	FK → `perfiles(id)`, NOT NULL	Gerente que crea la tarea
`id_capataz`	UUID	FK → `perfiles(id)`	Capataz asignado (nullable hasta asignación)
`id_estado`	INTEGER	FK → `estados_tarea(id_estado)`	Estado actual de la tarea
`fecha_baja`	DATE	—	Soft delete
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de creación
`updated_at`	TIMESTAMPTZ	DEFAULT now()	Última modificación

> ***Reglas de negocio:***
>
> - *Solo los gerentes pueden crear tareas.*
> - *`id_capataz` es inicialmente NULL y se asigna cuando el gerente elige un capataz.*
> - *Cuando `id_capataz` se establece, el estado pasa automáticamente a ASIGNADA.*
> - *`fecha_inicio` siempre debe ser ≤ `fecha_fin`.*
> - *La tarea puede tener múltiples trabajadores asignados a través de `tarea_trabajador`.*

3.5. TERRENO (entidad principal)

Representa las parcelas o terrenos de cultivo gestionados por los gerentes. Cada terreno pertenece a un único gestor (gerente) y puede albergar múltiples tareas a lo largo del tiempo.

Columna	Tipo	Restricciones	Descripción
`id_terreno`	SERIAL	**PK**	Identificador autoincremental
`nombre`	VARCHAR(200)	NOT NULL	Nombre identificativo del terreno
`ubicacion`	VARCHAR(255)	NOT NULL	Localización geográfica o dirección
`tipo_cultivo`	VARCHAR(100)	NOT NULL	Tipo de cultivo principal (ej: olivo, viñedo, cereal)
`estado`	VARCHAR(50)	DEFAULT 'ACTIVO'	Estado operativo del terreno
`id_gestor`	UUID	FK → `perfiles(id)`, NOT NULL	Gerente propietario/gestor

`fecha_baja`	DATE	—	Soft delete
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de registro

> ***Regla de negocio:*** *Un terreno pertenece exclusivamente a un gerente (* `id_gestor` *). El gerente solo puede crear tareas en terrenos que le pertenecen. El estado* `ACTIVO` / `INACTIVO` *controla si el terreno puede recibir nuevas tareas.*

3.6. GERENTE_CAPATAZ (relación N:M)

Tabla intermedia que implementa la relación **muchos-a-muchos** entre gerentes y capataces. Un gerente puede tener asignados varios capataces, y un capataz puede trabajar para varios gerentes.

Columna	Tipo	Restricciones	Descripción
`id_gerente`	UUID	**PK** compuesta, FK → `perfiles(id)`	Gerente que asigna
`id_capataz`	UUID	**PK** compuesta, FK → `perfiles(id)`	Capataz asignado
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de la asociación

> ***Clave primaria compuesta:*** *La combinación* `(id_gerente, id_capataz)` *es única, impidiendo duplicados. Eliminar una fila desvincula al capataz del gerente sin borrar ningún perfil.*

3.7. CAPATAZ_TRABAJADOR (relación N:M)

Tabla intermedia que implementa la relación **muchos-a-muchos** entre capataces y trabajadores. Permite que un capataz supervise a varios trabajadores y que un trabajador pueda estar asignado a varios capataces.

Columna	Tipo	Restricciones	Descripción
`id_capataz`	UUID	**PK** compuesta, FK → `perfiles(id)`	Capataz supervisor
`id_trabajador`	UUID	**PK** compuesta, FK → `perfiles(id)`	Trabajador supervisado
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de la asociación

> ***Cadena jerárquica:*** *Gerente → (gerente_capataz) → Capataz → (capataz_trabajador) → Trabajador. Esta cadena define quién puede asignar a quién en las tareas.*

3.8. TAREA_TRABAJADOR (relación N:M)

Tabla intermedia que vincula trabajadores con tareas específicas. Incluye campos adicionales que enriquecen la relación con información sobre quién realizó la asignación y si el trabajador la ha aceptado.

Columna	Tipo	Restricciones	Descripción
`id_tarea`	INTEGER	**PK** compuesta, FK → `tarea(id_tarea)`	Tarea asignada
`id_trabajador`	UUID	**PK** compuesta, FK → `perfiles(id)`	Trabajador asignado
`id_capataz`	UUID	FK → `perfiles(id)`	Capataz que realizó la asignación

`aceptada`	BOOLEAN	—	Si el trabajador ha aceptado la asignación
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de la asignación

> ***Regla de negocio:***
>
> - *Solo el capataz asignado a la tarea puede designar trabajadores.*
> - *El capataz solo puede asignar trabajadores que estén en su equipo (tabla `capataz_trabajador`).*
> - `aceptada` *indica si el trabajador ha confirmado la asignación.*

3.9. COMENTARIOS_TAREA (complementaria)

Sistema de comunicación interno que permite a los usuarios involucrados en una tarea dejar comentarios, facilitando la coordinación y el seguimiento de incidencias.

Columna	Tipo	Restricciones	Descripción
`id_comentario`	SERIAL	**PK**	Identificador autoincremental
`id_tarea`	INTEGER	FK → `tarea(id_tarea)`, NOT NULL	Tarea a la que pertenece el comentario
`id_autor`	UUID	FK → `perfiles(id)`, NOT NULL	Usuario que escribió el comentario
`contenido`	TEXT	NOT NULL	Texto del comentario
`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha y hora del comentario

> ***Regla de negocio:*** *Solo pueden comentar los usuarios vinculados a la tarea: el gerente que la creó, el capataz asignado y los trabajadores asignados. Esto se garantiza mediante políticas RLS.*

3.10. NOTIFICACIONES (complementaria)

Sistema de notificaciones en tiempo real que informa a los usuarios sobre eventos relevantes: asignaciones de tareas, cambios de estado, nuevos comentarios, etc. También dispara envíos de email mediante un trigger y la API de **Brevo (SendinBlue)**.

Columna	Tipo	Restricciones	Descripción
`id_notificacion`	SERIAL	**PK**	Identificador autoincremental
`id_usuario`	UUID	FK → `perfiles(id)`, NOT NULL	Usuario destinatario
`tipo`	VARCHAR(50)	NOT NULL	Tipo de notificación (ej: `TAREA_ASIGNADA`, `COMENTARIO_NUEVO`)
`mensaje`	TEXT	NOT NULL	Texto descriptivo de la notificación
`leida`	BOOLEAN	DEFAULT FALSE	Si el usuario ha leído la notificación
`id_referencia`	INTEGER	—	ID de la tarea u otra entidad relacionada (para navegación directa)

`created_at`	TIMESTAMPTZ	DEFAULT now()	Fecha de creación

> ***Automatización de emails:*** *Cuando se inserta una nueva notificación, un trigger PostgreSQL utiliza* `pg_net` *para hacer una petición HTTP a la API transaccional de Brevo, enviando un email al destinatario con los detalles de la notificación. Esto ocurre de forma asíncrona sin bloquear la operación original.*

---

4. Relaciones principales

El modelo contiene **15 claves foráneas** que establecen las relaciones entre las tablas. A continuación se detallan agrupadas por tipo de relación:

4.1. Relaciones N:1 (muchos a uno)

Estas son las relaciones más comunes, donde muchos registros de una tabla referencian un único registro de otra.

#	Origen (FK)	Destino (PK)	Cardinalidad	Descripción
1	`perfiles.id_rol`	`rol.id_rol`	N:1 obligatoria	Cada perfil tiene exactamente un rol
2	`tarea.id_terreno`	`terreno.id_terreno`	N:1 obligatoria	Cada tarea se ejecuta en un terreno específico
3	`tarea.id_gerente`	`perfiles.id`	N:1 obligatoria	Cada tarea es creada por un gerente
4	`tarea.id_capataz`	`perfiles.id`	N:1 **opcional**	Cada tarea puede tener un capataz asignado
5	`tarea.id_estado`	`estados_tarea.id_estado`	N:1 obligatoria	Cada tarea tiene un estado del ciclo de vida
6	`terreno.id_gestor`	`perfiles.id`	N:1 obligatoria	Cada terreno pertenece a un gerente
7	`comentarios_tarea.id_tarea`	`tarea.id_tarea`	N:1 obligatoria	Cada comentario pertenece a una tarea
8	`comentarios_tarea.id_autor`	`perfiles.id`	N:1 obligatoria	Cada comentario tiene un autor
9	`notificaciones.id_usuario`	`perfiles.id`	N:1 obligatoria	Cada notificación tiene un destinatario
10	`tarea_trabajador.id_capataz`	`perfiles.id`	N:1 **opcional**	Capataz que realizó la asignación

4.2. Relaciones N:M (muchos a muchos)

Resueltas mediante tablas intermedias con claves primarias compuestas:

#	Tabla intermedia	Tablas relacionadas	Descripción

11-12	`gerente_capataz`	`perfiles` ↔ `perfiles`	Un gerente puede tener N capataces; un capataz puede pertenecer a N gerentes
13-14	`capataz_trabajador`	`perfiles` ↔ `perfiles`	Un capataz puede supervisar N trabajadores; un trabajador puede tener N capataces
15	`tarea_trabajador`	`tarea` ↔ `perfiles`	Una tarea puede tener N trabajadores; un trabajador puede tener N tareas

4.3. Resumen visual de la cadena jerárquica

```
ADMIN (gestión global)
  │
  ├── GERENTE
  │     ├── posee → TERRENO (N:1)
  │     ├── crea  → TAREA (N:1)
  │     └── asigna ↔ CAPATAZ (N:M vía gerente_capataz)
  │           ├── supervisa ↔ TRABAJADOR (N:M vía capataz_trabajador)
  │           └── asigna trabajadores → TAREA (N:M vía tarea_trabajador)
  │
  └── Funcionalidades transversales
        ├── COMENTARIOS_TAREA → vinculados a TAREA + PERFILES
        └── NOTIFICACIONES → vinculados a PERFILES (+ trigger email Brevo)
```

---

5. Triggers y automatizaciones

El sistema utiliza varias funciones y triggers PostgreSQL que automatizan procesos críticos:

5.1. Trigger `handle_new_user()`

- **Evento:** `AFTER INSERT` en `auth.users`
- **Acción:** Crea automáticamente un registro en `perfiles` con los datos del usuario recién registrado
- **Campos copiados:** `id` (UUID), `email`, y los metadatos del formulario de registro (`nombre`, `apellidos`, `dni`, `tlf`, `direccion`, `id_rol`)
- **Importancia:** Garantiza que todo usuario autenticado tenga un perfil completo en la base de datos

5.2. Trigger de notificaciones por email

- **Evento:** `AFTER INSERT` en `notificaciones`
- **Acción:** Ejecuta una función que utiliza `pg_net` para hacer una petición HTTP POST a la API transaccional de **Brevo (SendinBlue)**
- **Datos enviados:** Email del destinatario, tipo de notificación, mensaje y enlace al sistema
- **Modo:** Asíncrono — la inserción de la notificación retorna inmediatamente sin esperar la respuesta del servicio de email

---

6. Políticas de seguridad (RLS)

Todas las tablas tienen habilitado **Row Level Security (RLS)** de PostgreSQL a través de Supabase. Las políticas garantizan que cada usuario solo acceda a los datos que le corresponden según su rol:

6.1. Desglose por rol

Rol	Operación	Alcance del acceso
ADMIN	SELECT, INSERT, UPDATE, DELETE	Acceso completo a **todas** las tablas y registros
GERENTE	SELECT	Sus propios terrenos, las tareas que ha creado, capataces vinculados a él (`gerente_capataz`), trabajadores de sus capataces
GERENTE	INSERT	Crear tareas y terrenos propios, vincular capataces
GERENTE	UPDATE	Modificar sus propias tareas y terrenos
CAPATAZ	SELECT	Tareas donde `id_capataz = auth.uid()`, trabajadores vinculados a él (`capataz_trabajador`)
CAPATAZ	UPDATE	Cambiar estado de tareas asignadas, asignar trabajadores
TRABAJADOR	SELECT	Tareas donde aparece en `tarea_trabajador`, sus propias notificaciones
TRABAJADOR	UPDATE	Marcar notificaciones como leídas

6.2. Implementación técnica

Las políticas utilizan la función `auth.uid()` de Supabase para obtener el UUID del usuario autenticado y compararlo con los campos de cada tabla. Ejemplo conceptual:

```
-- El gerente solo ve terrenos que le pertenecen
CREATE POLICY "Gerente: ver sus terrenos"
  ON terreno FOR SELECT
  USING (id_gestor = auth.uid());
```

Para las tablas intermedias (N:M), las políticas validan que el usuario esté presente en alguno de los lados de la relación, encadenando subconsultas que recorren la jerarquía Gerente → Capataz → Trabajador.

---

7. Patrón de eliminación lógica (Soft Delete)

Las tablas `perfiles`, `tarea` y `terreno` implementan un patrón de **soft delete** mediante el campo `fecha_baja`:

Tabla	Campo	Comportamiento
`perfiles`	`fecha_baja`	El usuario se desactiva pero su historial permanece en el sistema
`tarea`	`fecha_baja`	La tarea se archiva pero sigue siendo consultable
`terreno`	`fecha_baja`	El terreno se marca como inactivo pero conserva sus tareas históricas

Ventajas del patrón:

- **Integridad referencial:** Las FK que apuntan a registros «eliminados» siguen siendo válidas, evitando errores de cascada.
- **Auditoría:** Se conserva la fecha exacta de baja para trazabilidad.

- **Recuperación:** Un registro dado de baja puede reactivarse poniendo `fecha_baja = NULL`.
- **Filtrado:** Las consultas de la aplicación filtran por `fecha_baja IS NULL` para mostrar solo registros activos.

---

8. Restricciones e índices

8.1. Constraints UNIQUE

Tabla	Columna(s)	Propósito
`rol`	`nombre`	Sin roles duplicados
`estados_tarea`	`nombre`	Sin estados duplicados
`perfiles`	`dni`	Identificación fiscal única por persona
`perfiles`	`email`	Un solo registro por correo electrónico

8.2. Índices implícitos (creados por PostgreSQL)

PostgreSQL crea automáticamente índices B-tree para:

- Todas las columnas **PRIMARY KEY**
- Todas las columnas con constraint **UNIQUE**
- Las claves foráneas **no** generan índices automáticos, pero Supabase los añade en las tablas generadas por sus wizards

8.3. Timestamps automáticos

Las columnas `created_at` (DEFAULT `now()`) están presentes en todas las tablas, proporcionando trazabilidad temporal completa. La tabla `tarea` incluye además `updated_at` para registrar la última modificación.